

Rapport d'incident — Validation de resilience sur maillage de services

Deploiement Istio sur cluster Kubernetes local (kind) — mTLS, gestion du trafic et tolerance aux pannes

Anouar | 1er juillet 2026 | Environnement : kind v0.31, Kubernetes v1.35, Istio 1.30.2, k6 v2.0.0

1. Contexte et objectifs

Ce rapport documente la mise en place et la validation d'un maillage de services (service mesh) avec Istio sur un cluster Kubernetes local. L'objectif n'est pas de deployer un exemple standard, mais de verifier concretement trois proprietes attendues d'un maillage en production : le chiffrement et l'authentification automatiques du trafic interne (mTLS), la capacite a repartir et deplacer le trafic entre plusieurs versions d'un service (gestion du trafic), et la tolerance reelle a la panne d'un pod en cours de charge (resilience).

Chaque propriete est verifiee par une preuve d'execution (commande + resultat), et non par la seule presence d'un fichier de configuration applique. Un fichier YAML applique ne prouve pas qu'un comportement est actif ; seule une observation du trafic reel le prouve.

2. Architecture du banc de test

Un cluster kind a heberge le plan de controle Istio (istiod, passerelles d'entree/sortie) ainsi qu'un namespace applicatif **app**, avec injection automatique de sidecar activee. L'application se compose d'un service **backend** deploye en deux versions (v1 et v2, deux replicas chacune) et d'un pod **frontend** utilise comme client interne au maillage pour les tests.

3. Incident environnemental rencontre avant le lab

La creation du cluster kind a initialement echoue : le kubelet du noeud de control-plane redemarrant en boucle (290 tentatives observees) sans jamais atteindre l'etat healthy, faisant echouer *kubeadm init* par timeout. L'inspection des journaux du kubelet via *journalctl* a permis d'identifier la cause reelle, masquee par le message d'erreur generique de kubeadm : **inotify_init: too many open files**, provoquant l'echec du demarrage de cAdvisor et du watcher de certificats. La limite *fs.inotify.max_user_instances* du systeme hote (Fedora) etait insuffisante pour les besoins de kind. Le relevement de cette limite (sysctl) a resolu le probleme de maniere definitive.

Cet incident, bien qu'exterieur au perimetre du test de mesh, illustre une methode de diagnostic standard : ne pas se fier au message d'erreur de plus haut niveau, mais remonter aux journaux du composant reellement en echec.

4. Deploiement du maillage et injection des sidecars

Apres installation d'Istio (profil demo), l'etat du plan de controle a ete verifie avant tout deploiement applicatif.

```
anouar@192:~/k8s-lab$ kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istio-egressgateway-844b5dc4f7-krs4b 1/1     Running   0           70s
istio-ingressgateway-6d6855b4f5-fzp56 1/1     Running   0           70s
istiod-55bc777cb4-knz4x               1/1     Running   0          118s
```

Preuve 1 — istiod et les passerelles d'entree/sortie sont Running dans le namespace istio-system.

Le namespace applicatif a été étiqueté pour l'injection automatique de sidecar avant le déploiement des services. Chaque pod applicatif est attendu avec 2 conteneurs prêts (application + proxy Envoy).

```
anouar@192:~/k8s-lab$ kubectl get pods -n app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
backend-v1-5f7878cd7f-s5thw         2/2     Running   0           35s   10.244.0.10     mesh-lab-control-plane             <none>            <none>
backend-v1-5f7878cd7f-zlpxx         2/2     Running   0           35s   10.244.0.9      mesh-lab-control-plane             <none>            <none>
backend-v2-777c76d4f7-k2jxx         2/2     Running   0           35s   10.244.0.8      mesh-lab-control-plane             <none>            <none>
backend-v2-777c76d4f7-tp8nt         2/2     Running   0           35s   10.244.0.12     mesh-lab-control-plane             <none>            <none>
frontend-7448b4f684-2pctf           2/2     Running   0           35s   10.244.0.11     mesh-lab-control-plane             <none>            <none>
```

Preuve 2 — l'ensemble des pods du namespace app affichent READY 2/2, confirmant l'injection réelle du sidecar Envoy sur chaque pod (et non une simple présence du label sur le namespace).

5. Verification du chiffrement mutuel (mTLS)

Une politique **PeerAuthentication** en mode STRICT a été appliquée au namespace app. Pour vérifier que cette politique est réellement appliquée au trafic — et non simplement acceptée par l'API — deux requêtes identiques vers le service backend ont été émises depuis deux origines différentes : l'une depuis un pod hors maillage (namespace default, sans sidecar), l'autre depuis le pod frontend, à l'intérieur du maillage.

```
anouar@192:~/k8s-lab$ kubectl run tmp --image=curlimages/curl --rm -it --restart=Never -n default -- curl -m5 http://backend.app:8080
curl: (56) Recv failure: Connection reset by peer
pod "tmp" deleted from default namespace
pod default/tmp terminated (Error)
anouar@192:~/k8s-lab$ kubectl exec -n app deploy/frontend -- curl -m5 http://backend:8080
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0     0      0     0      0  0 --:--:-- --:--:-- --:--:--    0response from backend v1
100   25  100   25     0      0     0      0  0 --:--:-- --:--:-- --:--:-- 12500
```

Preuve 3 — la requête émise hors du maillage échoue ("Connection reset by peer"), le sidecar backend refusant une connexion en clair. La requête émise depuis le pod frontend, à l'intérieur du maillage, aboutit et retourne la réponse attendue. Ce contraste constitue la preuve que le mTLS est actif sur le trafic réel, et pas seulement configuré.

6. Gestion du trafic — repartition canary

Une **DestinationRule** définissant deux sous-ensembles (v1, v2) et une **VirtualService** repartissant le trafic à 90 % / 10 % ont été appliquées. La repartition a été vérifiée par comptage direct des réponses sur un échantillon de 20 requêtes émises depuis le pod frontend, plutôt que par simple lecture de la configuration.

```
destinationrule.networking.istio.io/backend created
virtualservice.networking.istio.io/backend created
anouar@192:~/k8s-lab$ kubectl exec -n app deploy/frontend -- sh -c 'for i in $(seq 1 20); do curl -s http://backend:8080; echo; done | sort | uniq -c'
20
17 response from backend v1
3 response from backend v2
```

Preuve 4 — sur 20 requêtes, 17 réponses proviennent de backend v1 et 3 de backend v2, soit une repartition conforme (à l'échantillonnage près) à la pondération 90/10 déclarée.

Une repartition 50/50 a également été testée et confirmée en session interactive ; le relevé correspondant n'a pas été conservé sous forme de capture et n'est donc pas présenté comme preuve dans ce rapport, conformément au principe de ne documenter que ce qui est effectivement démontrable.

7. Test de résilience sous charge réelle

L'objectif de ce test est de vérifier qu'une panne réelle d'un pod backend, survenant pendant une charge active, est absorbée par le maillage (retries, détection des pairs défaillants) sans interruption significative du service côté client. Le test a été mené en deux temps : une première tentative dont la méthodologie s'est révélée invalide, et une seconde, corrigée, dont le résultat est retenu comme preuve.

7.1 Tentative 1 — invalidée par une limite de la méthode de test

La charge a d'abord été générée avec k6 exécutée localement, le trafic étant acheminé vers le service backend via `kubectl port-forward`. Le pod backend v1 a été supprimé en cours de test.

```

execution: local
script: loadtest.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 1m30s max duration (incl. graceful stop):
 * default: 10 looping VUs for 1m0s (gracefulStop: 30s)

WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": read tcp 127.0.0.1:55540->127.0.0.1:8080: read: connection
reset by peer"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": read tcp 127.0.0.1:55534->127.0.0.1:8080: read: connection
reset by peer"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": read tcp 127.0.0.1:55538->127.0.0.1:8080: read: connection
reset by peer"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": EOF"
WARN[0019] Request Failed      error="Get \"http://localhost:8080\": read tcp 127.0.0.1:55554->127.0.0.1:8080: read: connection
reset by peer"

```

Preuve 5a — des l'instant de la suppression du pod, k6 rapporte des echecs en cascade (EOF, connection reset by peer).

```

Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
E0701 19:21:23.795798 1259162 portforward.go:522] "Unhandled Error" err="an error occurred forwarding 8080 -> 8080: error forwarding port 8080 to pod 286
2894df43403d0ff335313805abc7d79f66295e71f3070b53d5e531ab6e62f, uid : network namespace for sandbox \"2862894df43403d0ff335313805abc7d79f66295e71f3070b53d
5e531ab6e62f\" is closed"
Handling connection for 8080
E0701 19:21:24.007445 1259162 portforward.go:455] "Unhandled Error" err="error creating error stream for port 8080 -> 8080: write tcp 127.0.0.1:33178->12
7.0.0.1:42853: write: broken pipe"
Handling connection for 8080
Handling connection for 8080
E0701 19:21:24.008951 1259162 portforward.go:455] "Unhandled Error" err="error creating error stream for port 8080 -> 8080: write tcp 127.0.0.1:33178->12
7.0.0.1:42853: write: broken pipe"
E0701 19:21:24.009097 1259162 portforward.go:455] "Unhandled Error" err="error creating error stream for port 8080 -> 8080: write tcp 127.0.0.1:33178->12
7.0.0.1:42853: write: broken pipe"

```

Preuve 5b — le journal du port-forward confirme la cause réelle : le tunnel lui-meme se ferme ("network namespace for sandbox ... is closed", "broken pipe"), car kubectl port-forward est lie au pod cible et non au service. La suppression du pod a rompu l'outil de test, pas seulement le pod.

```

anouar@192:~/k8s-lab$ kubectl delete pod -n app -l version=v1
pod "backend-v1-5f7878cd7f-s5thw" deleted from app namespace
pod "backend-v1-5f7878cd7f-zlpxx" deleted from app namespace
anouar@192:~/k8s-lab$

```

Preuve 5c — commande de suppression du pod v1 ayant declenche l'incident de tunnel.

TOTAL RESULTS

```
checks_total.....: 2897 48.258894/s
checks_succeeded...: 31.10% 901 out of 2897
checks_failed.....: 68.89% 1996 out of 2897
```

```
X status is 200
  L 31% - ✓ 901 / X 1996
```

HTTP

```
http_req_duration.....: avg=5.42ms min=0s med=0s max=1.27s p(90)=6.31ms p(95)=8.42ms
{ expected_response:true }...: avg=5.47ms min=606.2µs med=4.73ms max=31.94ms p(90)=9.83ms p(95)=11.98ms
http_req_failed.....: 68.89% 1996 out of 2897
http_reqs.....: 2897 48.258894/s
```

EXECUTION

```
iteration_duration.....: avg=207.14ms min=200.31ms med=202.12ms max=1.47s p(90)=207.72ms p(95)=210.18ms
iterations.....: 2897 48.258894/s
vus.....: 10 min=10 max=10
vus_max.....: 10 min=10 max=10
```

NETWORK

```
data_received.....: 169 kB 2.8 kB/s
data_sent.....: 65 kB 1.1 kB/s
```

Preuve 5d — resultat final de cette tentative : 68,89 % d'echecs sur 2897 requetes. Ce chiffre ne mesure pas la resilience du maillage : il mesure la rupture de l'outil de mesure lui-meme. Resultat ecarte pour cette raison.

7.2 Correction de methode

Le probleme identifie est que *kubectrl port-forward* etablit un tunnel vers un pod precis et non vers le service Kubernetes : la suppression du pod cible casse le canal de test avant meme que l'equilibrage de charge du maillage n'entre en jeu. La correction consiste a executer le generateur de charge (k6) directement dans un pod du namespace app, avec injection de sidecar activee, interrogeant le service backend par son nom DNS interne. Le trafic passe alors reellement par les proxys Envoy et par l'equilibrage de charge du maillage, comme le ferait un client applicatif reel.

7.3 Tentative 2 — test valide, execute a l'interieur du maillage

Le pod backend v1 a de nouveau ete supprime en cours de charge, cette fois avec k6 execute comme Job Kubernetes dans le namespace app.

```
anouar@192:~/k8s-lab$ kubectl delete pod -n app -l version=v1
pod "backend-v1-5f7878cd7f-88xfd" deleted from app namespace
pod "backend-v1-5f7878cd7f-qgl6x" deleted from app namespace
anouar@192:~/k8s-lab$
```

Preuve 6a — suppression des pods backend v1 pendant l'execution du test de charge interne au maillage.

```

default [ 100% ] 10 VUs 1m00.0s/1m0s

TOTAL RESULTS

checks_total.....: 2911 48.356426/s
checks_succeeded...: 95.25% 2773 out of 2911
checks_failed.....: 4.74% 138 out of 2911

X status is 200
  95% - ✓ 2773 / ✗ 138

HTTP
http_req_duration.....: avg=5.42ms min=372.59µs med=4.44ms max=167.5ms p(90)=9.39ms p(95)=14.97ms
{ expected_response:true }...: avg=5.51ms min=751.05µs med=4.51ms max=53.38ms p(90)=9.63ms p(95)=15.16ms
http_req_failed.....: 4.74% 138 out of 2911
http_reqs.....: 2911 48.356426/s

EXECUTION
iteration_duration.....: avg=206.44ms min=200.71ms med=205.48ms max=368.51ms p(90)=210.5ms p(95)=216.04ms
iterations.....: 2911 48.356426/s
vus.....: 10 min=10 max=10
vus_max.....: 10 min=10 max=10

NETWORK
data_received.....: 676 kB 11 kB/s
data_sent.....: 198 kB 3.3 kB/s

running (1m00.2s), 00/10 VUs, 2911 complete and 0 interrupted iterations
default ✓ [ 100% ] 10 VUs 1m0s

```

Preuve 6b — resultat final : 2911 requetes emises, 95,25 % de succes (2773), 4,74 % d'echec (138) durant la fenetre de suppression et de redemarrage des pods v1. Le taux d'echec residuel correspond a la fenetre normale de drainage des connexions et de detection de defaillance, avant la reprise complete du trafic sur les pods sains.

8. Synthese des resultats

Propriete testee	Methode de verification	Resultat
Plan de controle Istio	kubectrl get pods -n istio-system	Conforme
Injection de sidecar	kubectrl get pods -n app -o wide	Conforme (2/2 Ready)
mTLS strict	Requete hors/dans le maillage	Confirme (rejet / succes)
Repartition de trafic 90/10	Comptage sur 20 requetes	17/3 observe
Resilience a la panne de pod (mesure invalide)	k6 local + port-forward	68,89 % d'echec (methode invalidee)
Resilience a la panne de pod (mesure valide)	k6 in-cluster (Job, sidecar)	95,25 % de succes (2911 requetes)

9. Conclusion

Les trois proprietes visees ont ete verifiees par observation directe du comportement du trafic, et non par simple inspection de configuration. Le point le plus significatif de cet exercice n'est pas le resultat final de 95,25 % de succes sous panne de pod, mais la demarche qui y a mene : une premiere mesure a produit un resultat en apparence alarmant (68,89 % d'echec), et son analyse a revele que l'outil de mesure, et non le systeme teste, etait en cause. Corriger la methode avant de tirer une conclusion sur le systeme est la partie du travail qui a le plus de valeur ici.